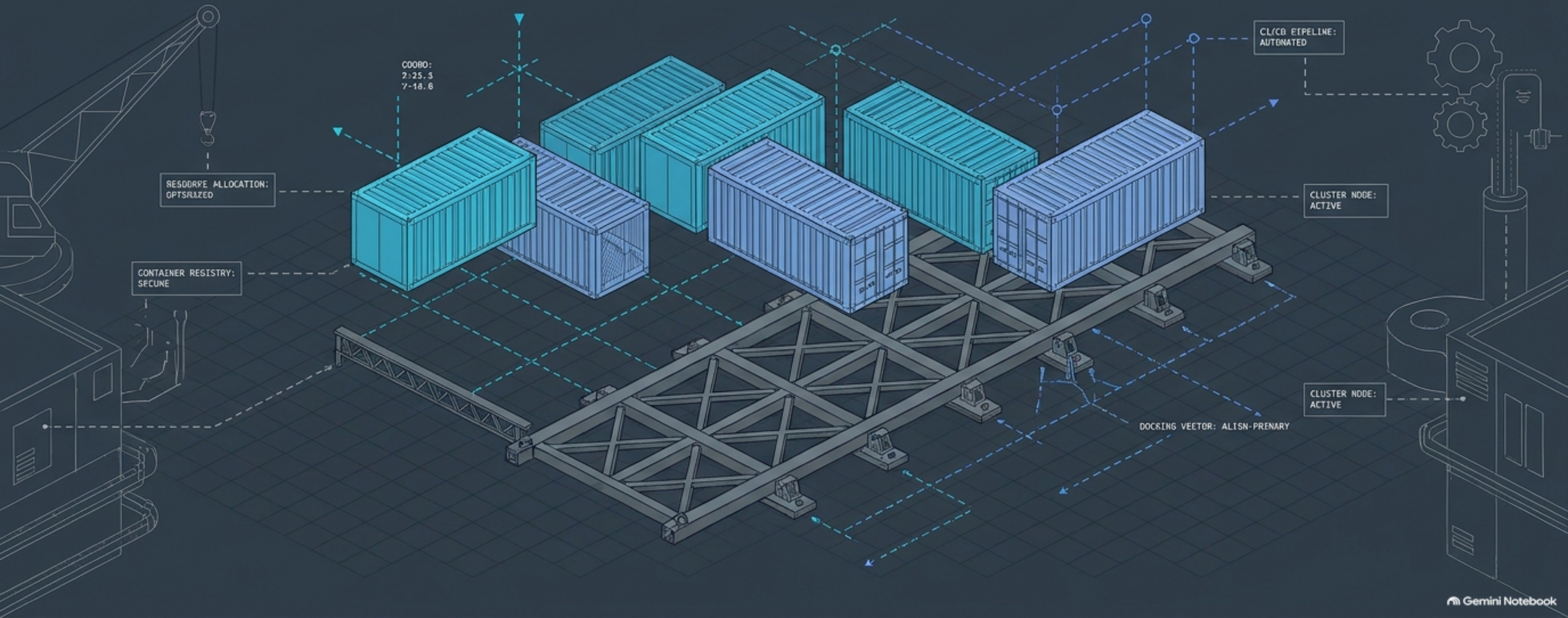
# ORQUESTRANDO O CAOS: FUNDAMENTOS DE KUBERNETES, DOCKER E CI/CD

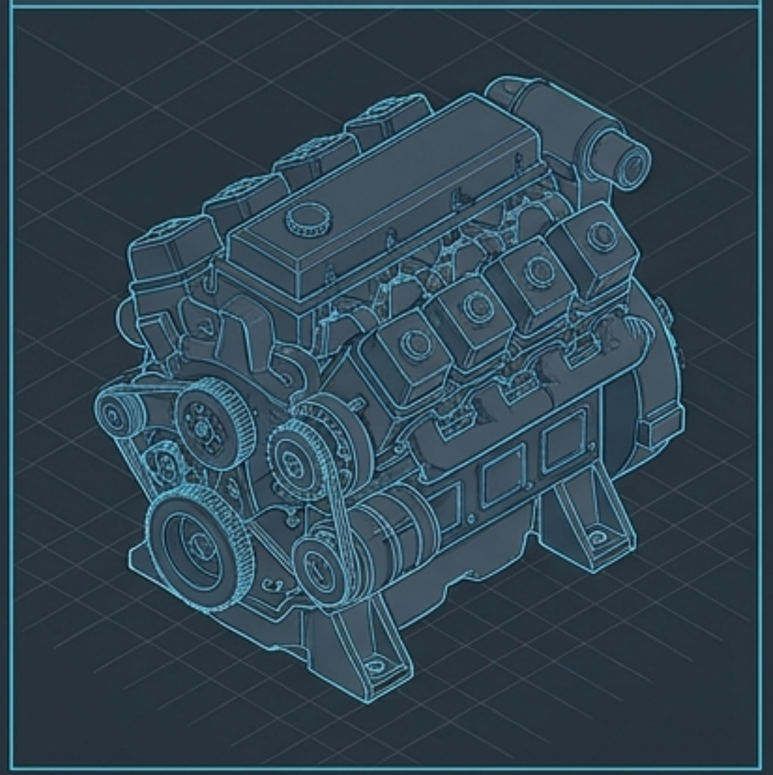
TERMINAL EXEC: /usr/bin/docker-compose up --build; kubectl apply -f deployment.yaml; git pipeline run --prod




Gemini Notebook

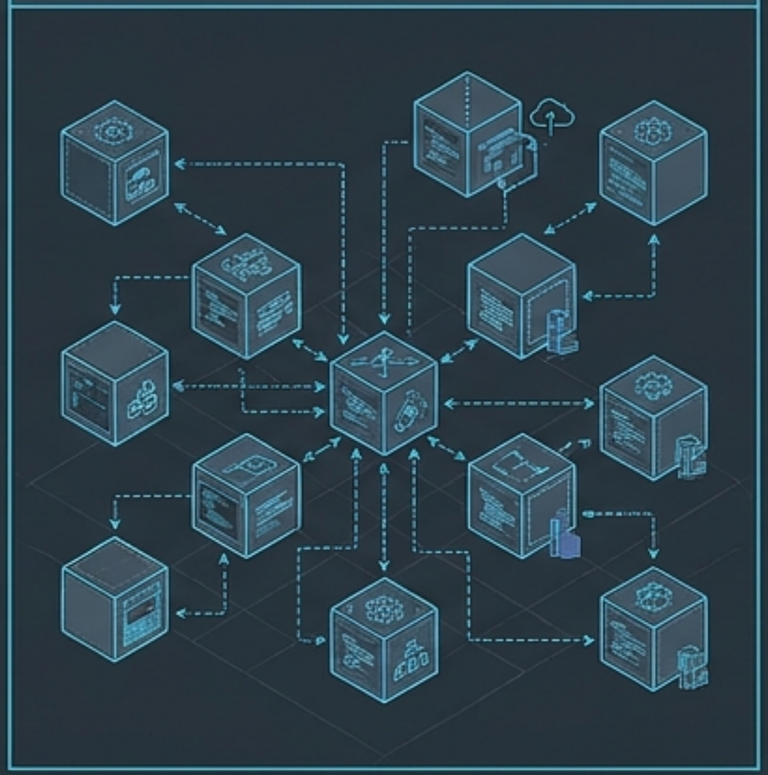
# A evolução da arquitetura: do monólito aos contêineres portáteis

## O Passado (Monólito)



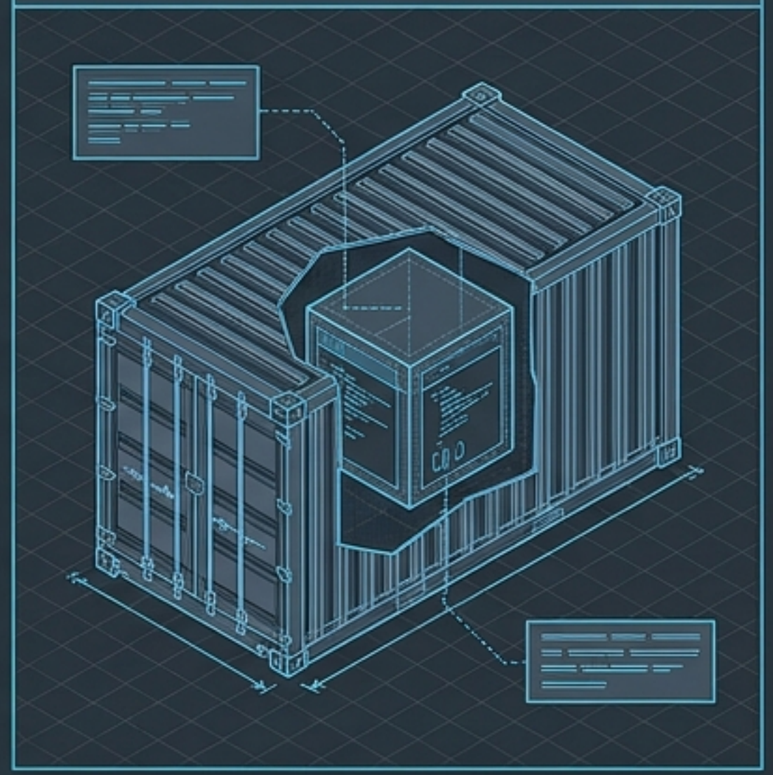
Uma única estrutura rígida onde todos os componentes são interdependentes.

## O Presente (Microserviços)



Pequenos serviços independentes, comunicando-se via APIs, com deploy e escalonamento separados.

## O Veículo (Contêiner)



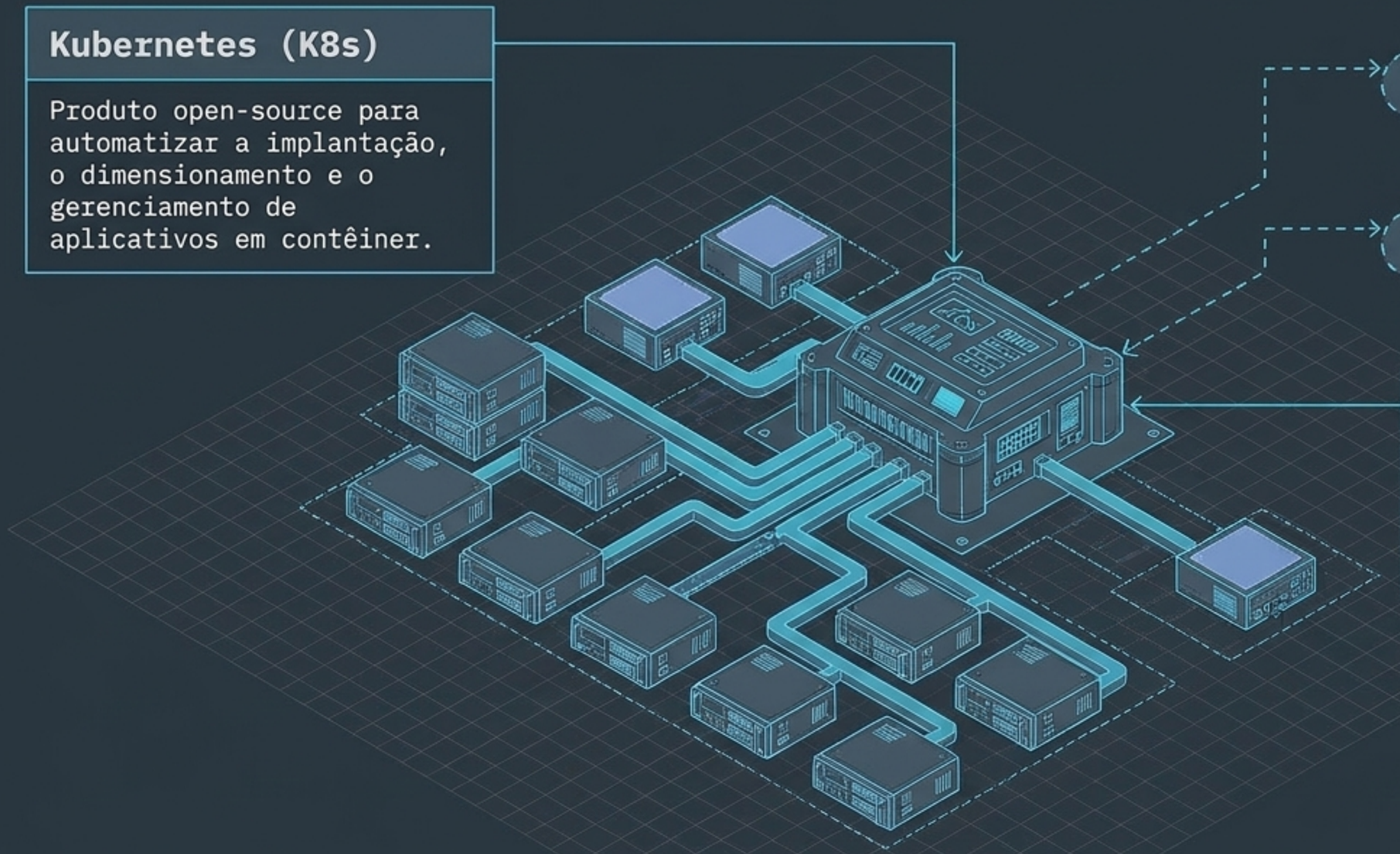
Tecnologia que reúne o aplicativo e seus arquivos necessários para o tempo de execução.
Portátil: pode ser movido e executado em qualquer contexto.

Gemini Notebook

# Um único sistema formado por múltiplos computadores

## Kubernetes (K8s)

Produto open-source para automatizar a implantação, o dimensionamento e o gerenciamento de aplicativos em contêiner.



**Amazon EKS:** Serviço gerenciado para rodar K8s na AWS.

**Google GKE:** Plataforma gerenciada de K8s no GCP.

## Cluster

Computadores interligados que trabalham em conjunto e podem ser considerados um único sistema coeso.

Gemini Notebook

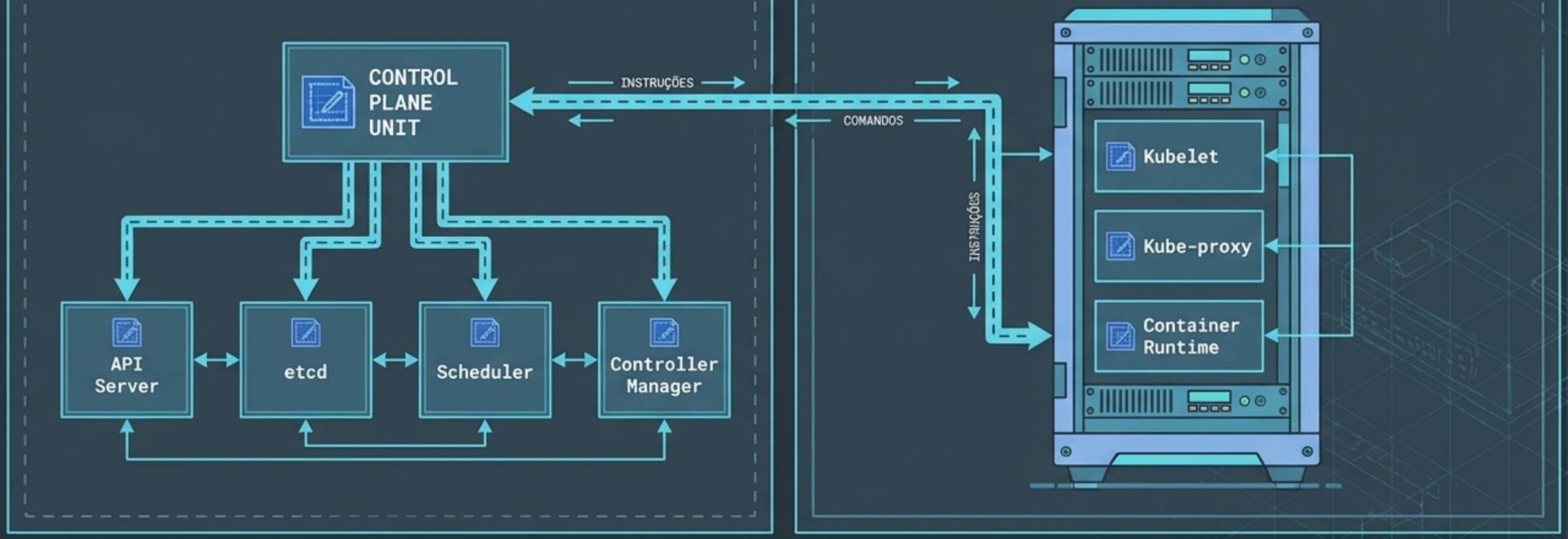
# A anatomia do cluster: Cérebro e Músculos

## O Cérebro (Servidor de Gerenciamento / Control Plane)

Administra os nós de processamento e os Pods no cluster.

## Os Músculos (Workers / Nós de Processamento)

Hospeda os objetos presentes no cluster, incluindo pods, serviços e controladores de replicação.

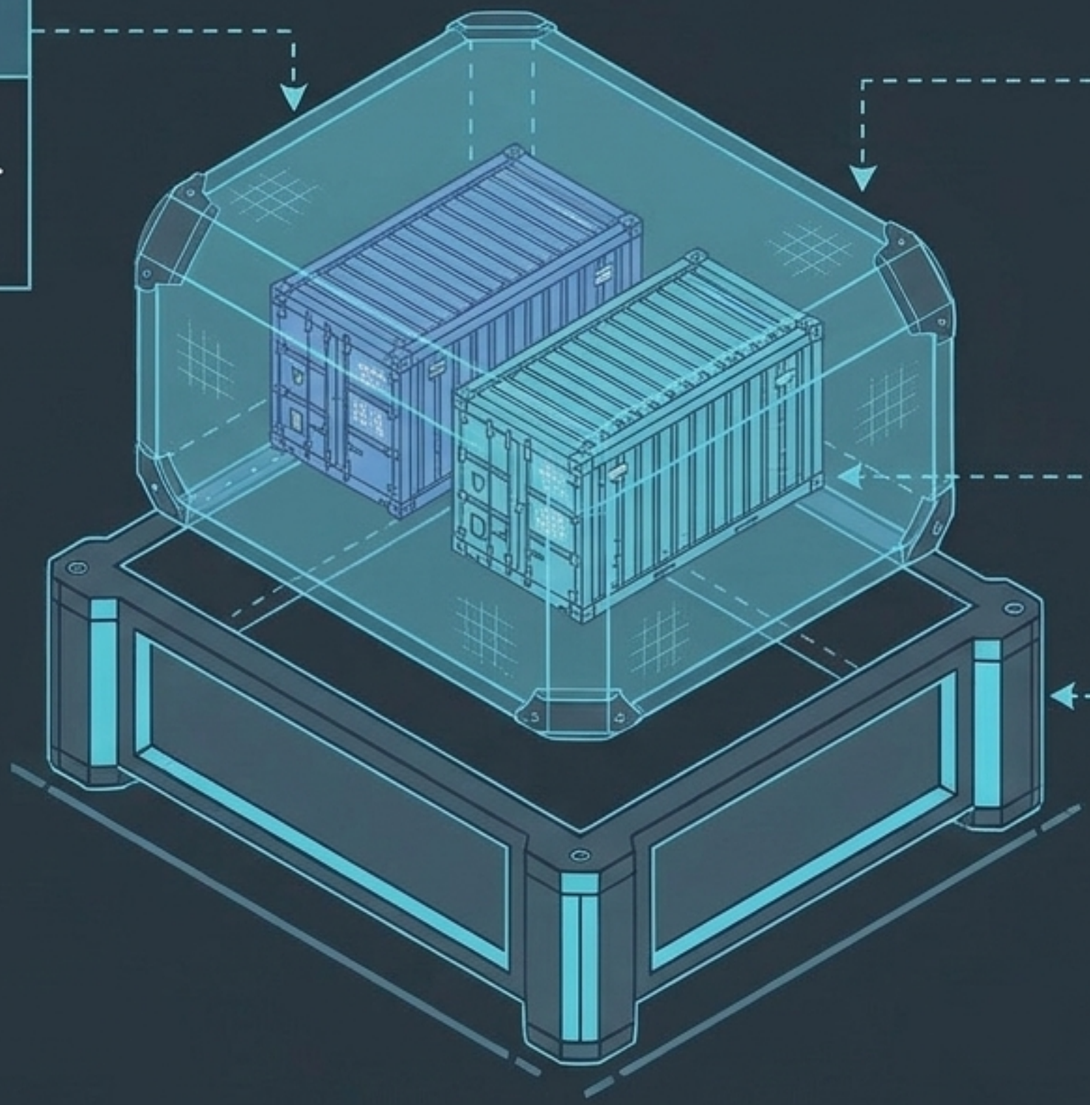



Gemini Notebook

# O Pod atua como um casulo efêmero para os contêineres

**Pod**

A menor unidade implantável do Kubernetes. Compartilha a mesma rede e armazanamento.



**Pod**

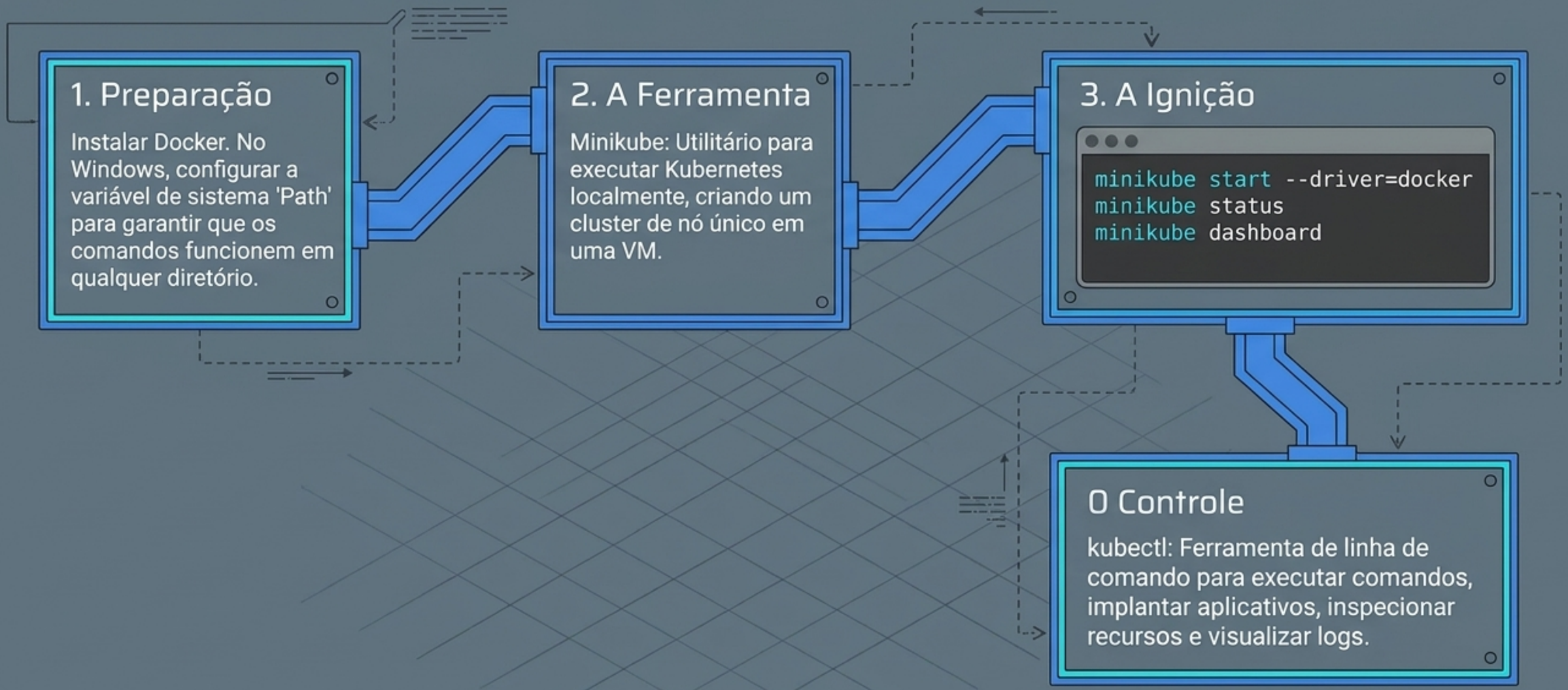
Ciclo de vida efêmero.

**Contêineres**

**Worker Node**

Gemini Notebook

# Laboratório de bolso: rodando Kubernetes localmente



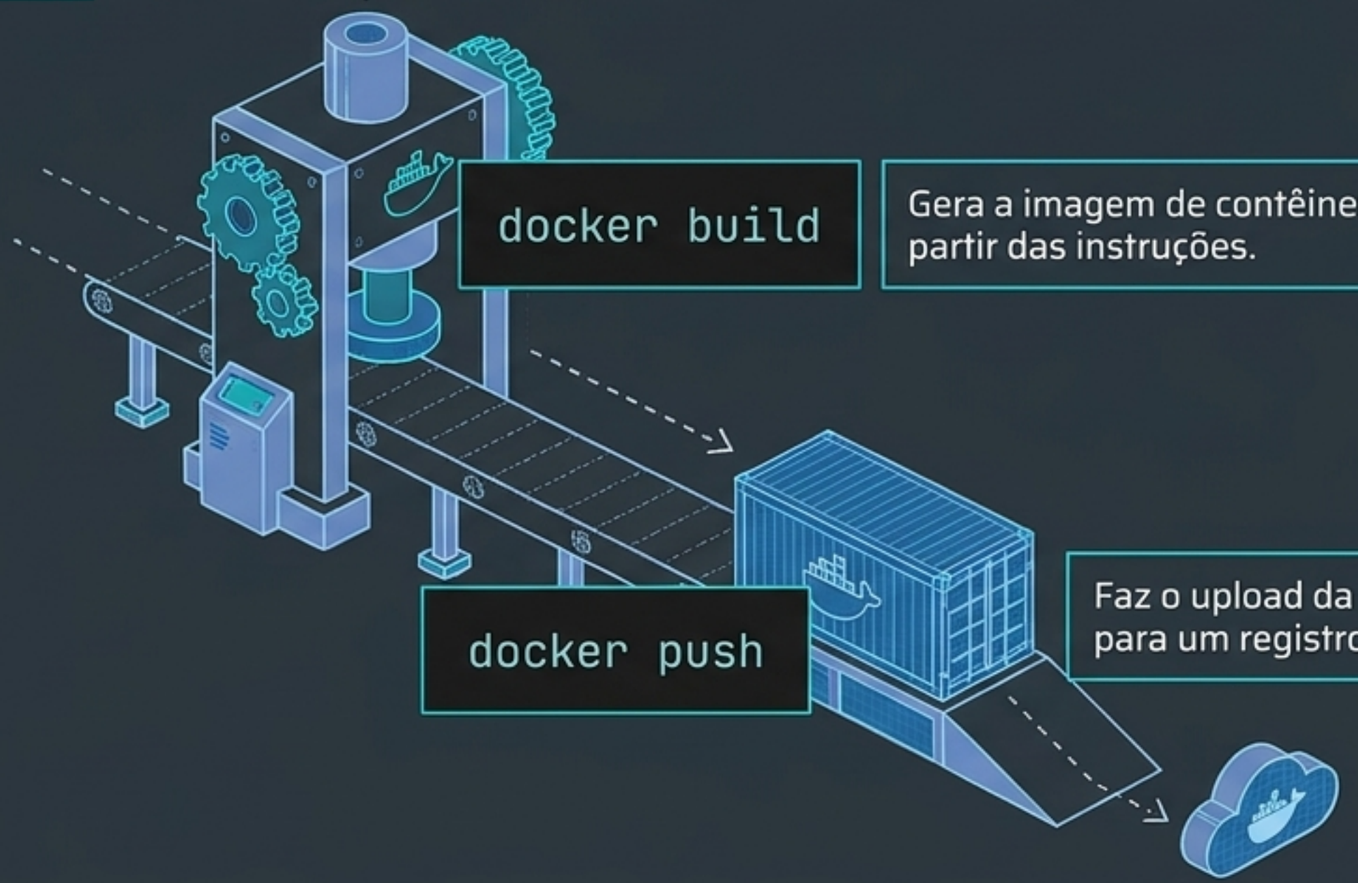

Gemini Notebook

# A linha de montagem da imagem de contêiner

**Dockerfile**

Arquivo de texto usado para armazenar as instruções de construção.

Parâmetro WORKDIR:
Define o diretório padrão de trabalho dentro do contêiner.



**`docker build`**

Gera a imagem de contêiner a partir das instruções.

**`docker push`**

Faz o upload da imagem gerada para um registro.

Gemini Notebook

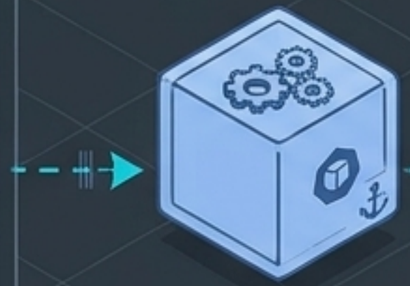
# Colocando no ar: do manifesto YAML ao tráfego de rede

## Passo 1: O Manifesto

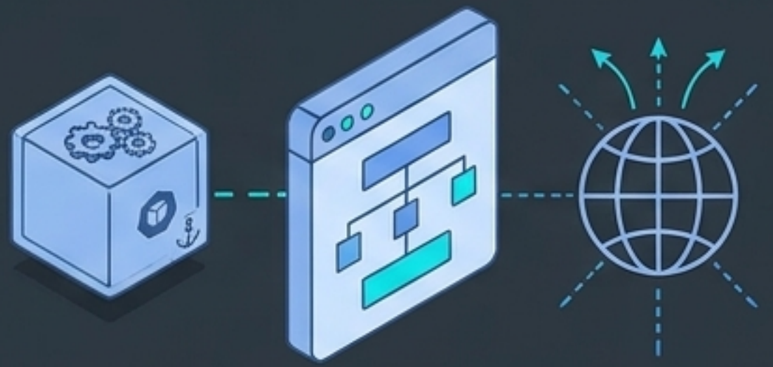



```
kubectl apply -f pod.yml
```

**Atenção:** O comando exige o nome exato do arquivo (pod.yml).



## Passo 2: O Serviço



Um Service é necessário para conexões via código ou para gerar um IP fixo de acesso externo ao Deployment.

```
kubectl expose deployment app-html
--type=LoadBalancer --name app-html
--port 80
```

Gemini Notebook

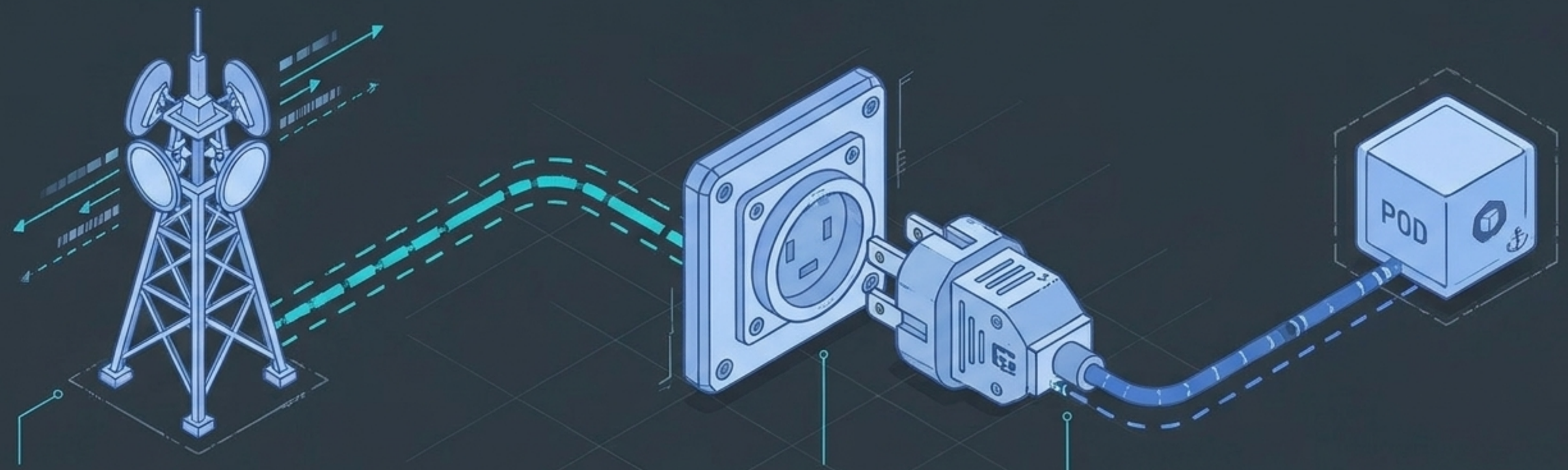
# O problema da memória: se o Pod morre, os dados desaparecem




O ciclo de vida do Pod é efêmero. No Kubernetes, o armazenamento local vive e morre com a aplicação. Como mantemos um banco de dados vivo em um ambiente volátil?

Gemini Notebook

# A solução persistente: conectando Pods ao armazenamento externo




**A Rede (NFS)**

Serviço que possibilita o compartilhamento remoto de arquivos via rede no modelo cliente/servidor.

**A Tomada (PV - Persistent Volume)**

Plugin de volume com ciclo de vida independente dos Pods. A API abstrai a infraestrutura.

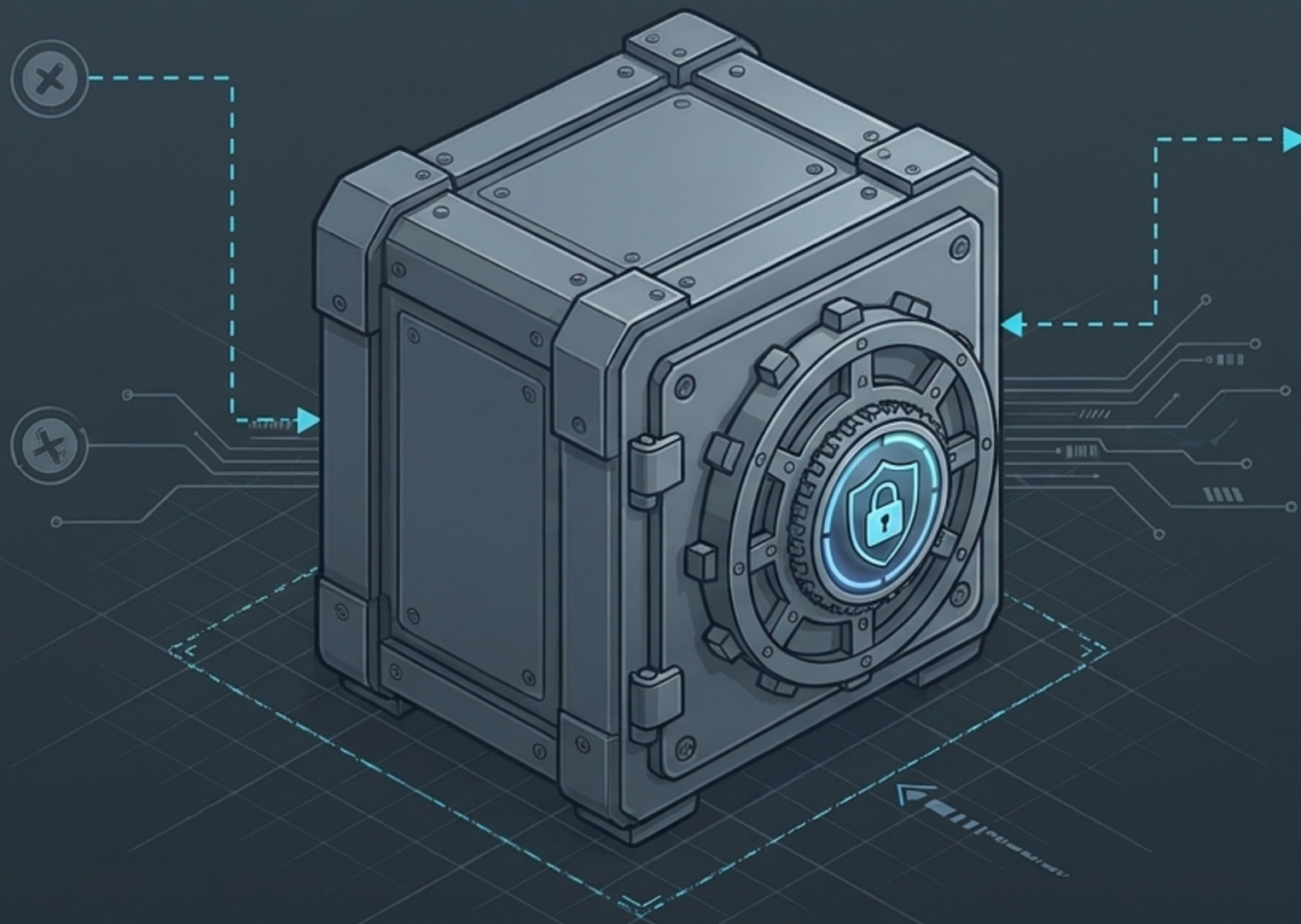
**O Pino (PVC - Persistent Volume Claim)**

Requisição de armazenamento feita pelo usuário, com tamanho e modos de acesso, que se conecta ao PV.

Regra de Acesso: ReadWriteMany permite que múltiplos nós leiam e gravem simultaneamente.

Gemini Notebook

# O cofre do cluster: protegendo dados sensíveis



## Recurso: Secret

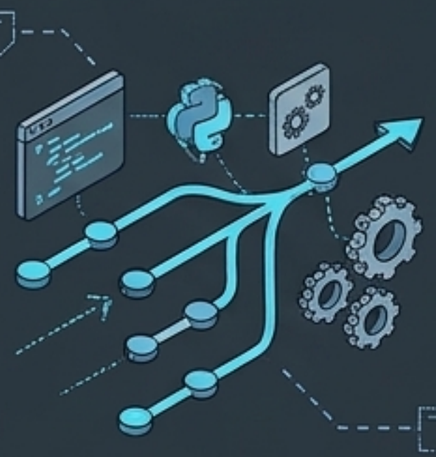
O recurso projetado para armazenar e gerenciar senhas, tokens e chaves privadas.

Injeta credenciais nos Pods e Deployments de forma segura, garantindo que nunca fiquem expostas diretamente no código-fonte ou nas imagens de contêiner.

Gemini Notebook

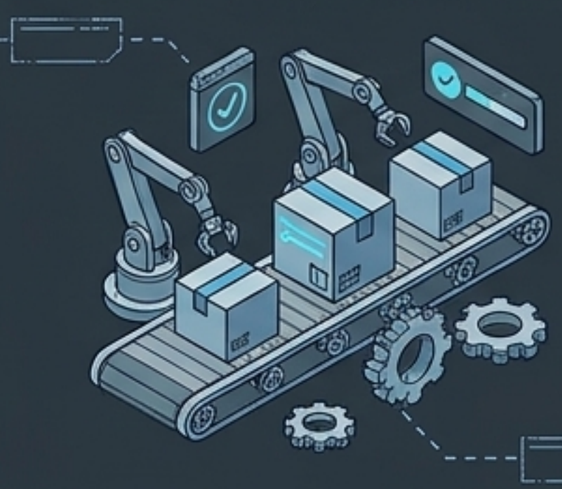
# A linha do tempo DevOps: Desmistificando a automação

## 1. Integrar (CI - Integração Contínua)



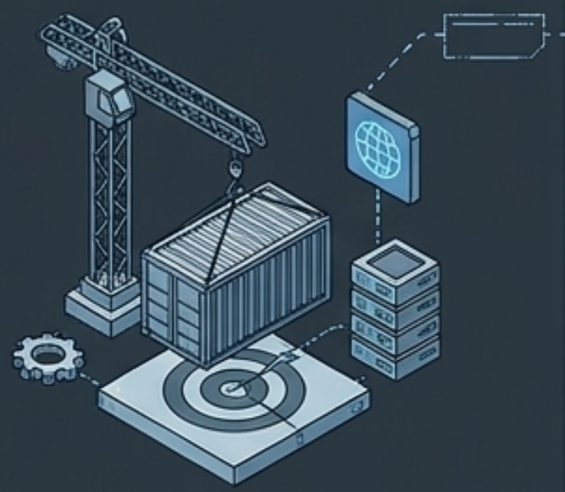
Prática de integrar alterações de código frequentemente em um repositório central, com builds e testes automatizados.

## 2. Entregar (CD - Entrega Contínua)



Prática que automatiza a criação, os testes e a preparação do software para liberação em produção.

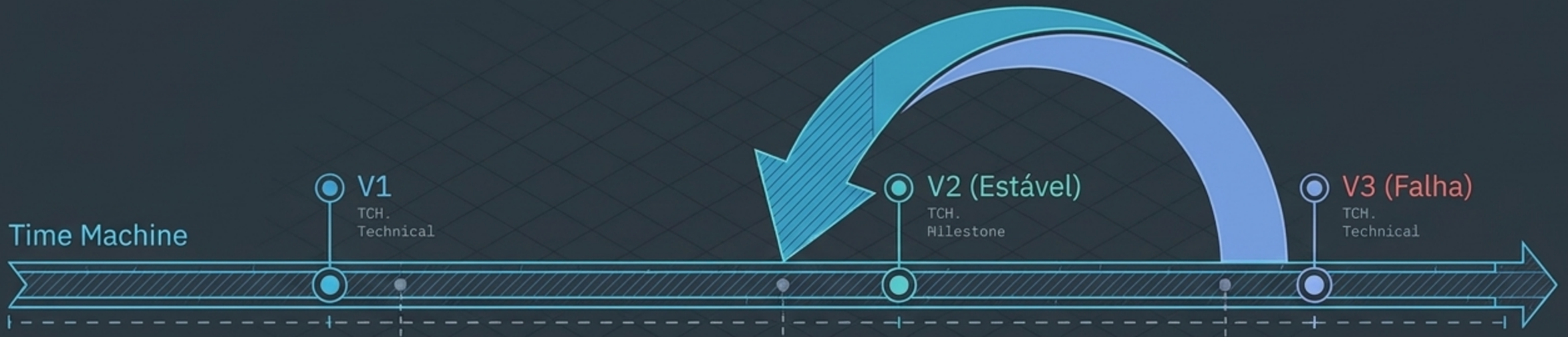
## 3. Implantar (Deploy)



Ação final operacional de mover o software de um ambiente controlado para outro ambiente definitivo.

Gemini Notebook

# Controle do tempo: inspecionando e revertendo implantações




1

## Gravando a História

```
kubectl apply --record
```

A flag mantém o histórico dos comandos de implantação.

2

## Consultando o Passado

```
kubectl rollout history
deployment DIO
```

Audita todas as versões registradas do Deployment.

3

## O Salva-Vidas (Rollback)

```
kubectl rollout undo
deployment httpd
```

Força o sistema a reverter imediatamente para a última versão operante.

Gemini Notebook

# Painel de Controle de Sobrevivência (Comandos Essenciais)

## Investigar e Inspecionar

```
kubectl cluster-info
```
Estado geral do cluster.

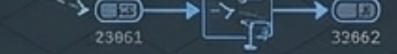
```
kubectl get nodes
```
Listar todos os nós.

```
kubectl get pod
```
Visualizar pods em execução.

## Intervir Diretamente

```
kubectl exec
```
Executar aplicações no Pod.

```
kubectl port-forward
pod/bd-mysql 3306:3306
```
Mapear porta direta.




## Gerenciar Ciclo de Vida

```
kubectl apply -f pod.yml
```
Criar via manifesto.

```
kubectl delete pod app-html
```
Destruição cirúrgica.

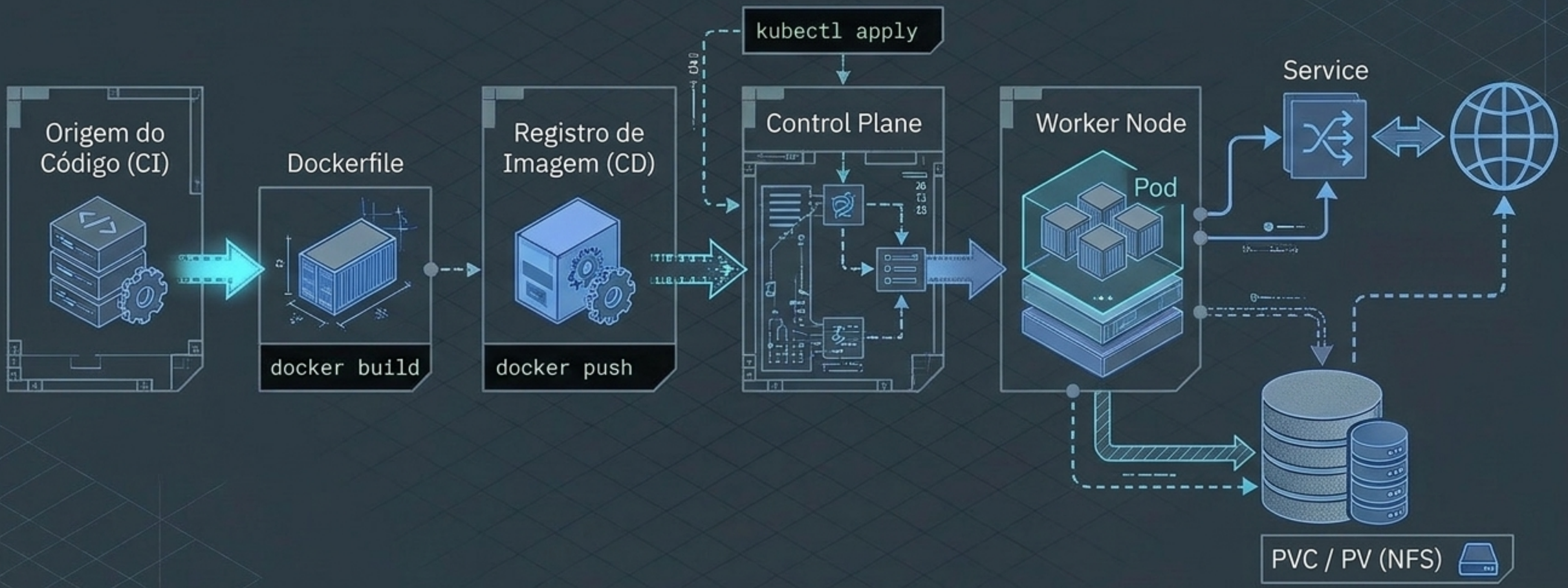
## Controle de Tráfego

```
kubectl expose
```
Criar Service / LoadBalancer.



Gemini Notebook

# O Mapa Sistêmico: Do código-fonte à resiliência em escala




**A arquitetura não se trata apenas de executar código, mas de desenhar sistemas imunes ao caos.**

Gemini Notebook